University of British Columbia

Electrical and Computer Engineering

ELEC391 – Summer 2026

Instructor: Dr. Saloome Motavas

Section: 941

Self-Balancing Robot - Final Report

Group 11

Group Member Name	Group Member Student #	Contribution to Project (%)
Peter Gao		100%
Tony Zhao		100%
Echo Zhou		100%
Rita Chang		100%

Date of Submission: Jun 21th 2026

Table of Content

1. System Overview..... 3
2. Final Design and Implementation..... 5
2.1 Mechanical Design..... 5
2.1.1 Chassis Structure..... 5
2.1.2 Component Placement..... 5
2.1.3 Center-of-Mass Considerations..... 6
2.1.4 Photos and Diagrams..... 7
2.2 Electrical Design..... 7
2.2.1 Circuit schematics..... 7
2.2.2 Wiring diagram..... 8
2.2.3 Power distribution..... 9
2.2.4 Sensors and actuators..... 9
2.3 Software and Control Design..... 10
2.3.1 Software architecture..... 10
2.3.2 Sensor processing and filtering..... 11
2.3.3 Control algorithms..... 11
2.3.4 BLE integration..... 13
2.3.5 Final control gains and parameters..... 14
3. Design Evolution and Engineering Decisions..... 15
3.1 Midpoint Demo Performance..... 15
3.2 IMU Mounting and Calibration..... 15
3.3 Control Structure Update..... 16
3.4 Motor and BLE Improvements..... 17
3.5 Mechanical Adjustment..... 17
3.6 Final Design Direction..... 18
4. Testing, Tuning, and Performance Analysis..... 18
4.1 Balance and Sensor Stability Test..... 18
4.2 Motor Deadband and Position Accuracy Test..... 19
4.3 Turning and Ramp Performance Test..... 20
5. Challenges and Troubleshooting..... 22
5.1 Stationary Drift and Inconsistent Balancing..... 22
5.2 Motor Mismatch and Asymmetric Turning..... 23
5.3 Steep Ramp Descent Failure..... 24
6. Conclusions..... 25
6.1 Final System Performance..... 25
6.2 Technical Achievements..... 25
6.3 Important Lessons Learned..... 26

7. Future Improvements..... 26
7.1 Improved Downhill Ramp Control..... 26
7.2 Direction-Specific Turning Calibration..... 26
7.3 Better Mechanical Stability..... 27
7.4 Battery Voltage and Motor Compensation.....27
7.5 More Systematic Data Logging..... 27

1. System Overview

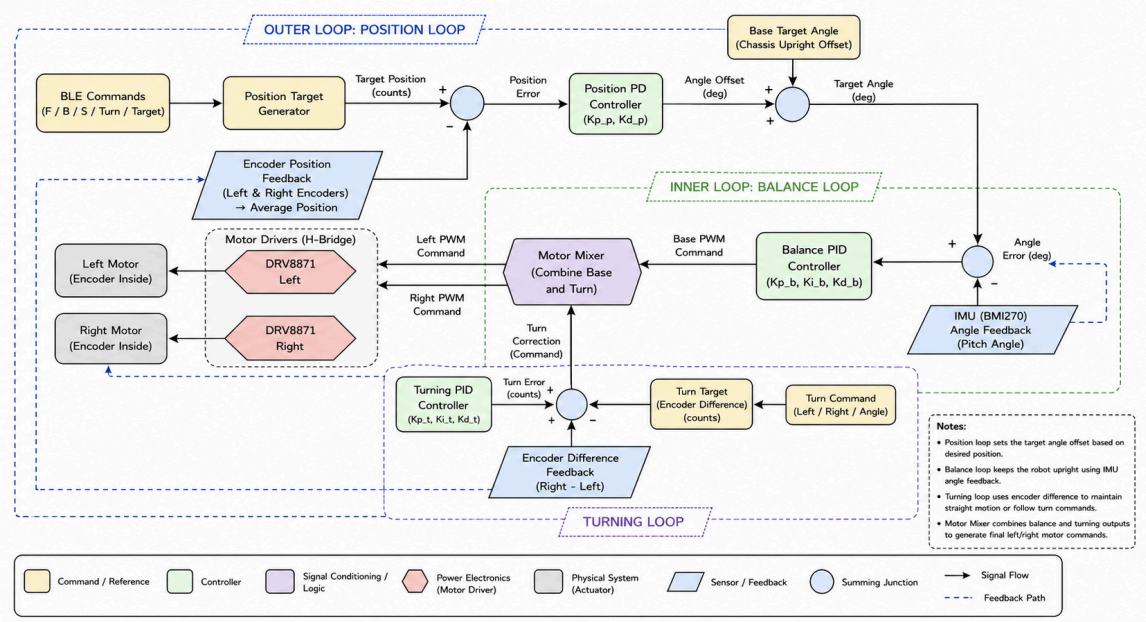


Figure 1. Block diagram of the robot system

The final robot system is a two-wheel self-balancing robot controlled by an Arduino Nano 33 BLE Sense. As shown in Figure 1, the system includes the IMU, wheel encoders, Arduino controller, DRV8871 motor drivers, DC motors, power supply, and BLE communication module. The IMU provides tilt feedback, while the encoders measure wheel position and movement.

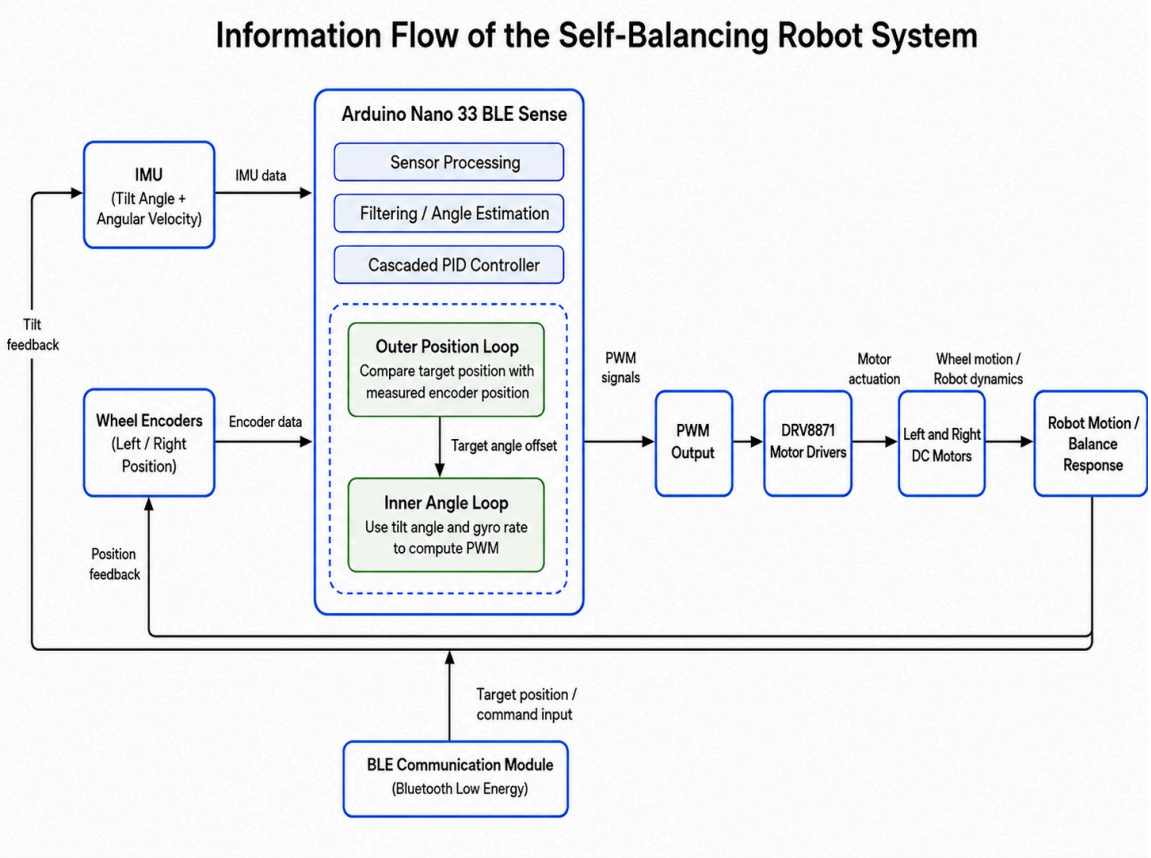


Figure 2. Flowchart of Information

The information flow follows a closed-loop control structure as shown in Figure 2. The IMU measures the robot's tilt angle and angular velocity, and the encoders measure the left and right wheel positions. These sensor signals are processed by the Arduino and used by the cascaded PID controller. The outer position loop compares the target position with the measured encoder position and generates a small target angle offset. This target angle is then sent to the faster inner angle loop, which calculates the PWM output needed to keep the robot balanced. The PWM signals are sent to the motor drivers, which control the left and right motors.

The balancing mechanism is based on the interaction between the position loop and the angle loop. The position loop determines how much the robot should tilt to reach or hold a target position, while the angle loop keeps the robot upright by continuously adjusting the motor output. This allows the robot to balance and move toward commanded positions at the same time.

BLE communication is used as a wireless command interface. The Arduino advertises as a BLE device and receives commands from an external device, such as a phone or computer. These commands can update the target position, move the robot forward or backward, select preset positions, or hold the current position. BLE polling is performed periodically so that it does not block the main control loop.

Overall, the final robot can balance on two wheels, hold its position using encoder feedback, move to target positions, and receive wireless BLE commands. In performance testing, the robot was also able to remain stable on both 5° and 10° ramps. When balancing upright in place, the position oscillation was maintained within approximately ±3 cm. For turning performance, the robot was able to complete a 90° turn with a heading error within 5°, showing that the yaw correction and heading control were effective.

2. Final Design and Implementation

2.1 Mechanical Design

2.1.1 Chassis Structure

The robot uses a two-wheel self-balancing chassis, with one DC motor and wheel mounted on each side of the lower frame. This design allows the robot to behave like an inverted pendulum, where the body naturally tends to fall forward or backward unless the wheels move to correct the tilt. The motors are mounted near the bottom of the chassis so that the wheel axle acts as the main pivot point of the robot. The frame was kept as rigid as possible to reduce vibration and prevent movement between the chassis and sensors. This is important because any loose mechanical connection can affect the measured tilt angle and make the balancing control less stable.

2.1.2 Component Placement

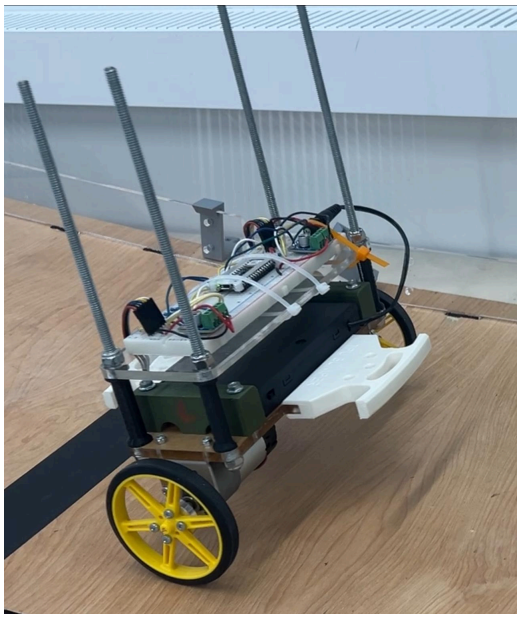


Figure 3. The Placement of the Component

As shown in Figure 3, the robot body is arranged as a two-layer structure above the motor base. The two DC motors are mounted at the lowest part of the robot, with one motor connected to each wheel on the left and right sides. Above the motors, the main chassis is divided into an upper layer and a lower layer. The upper layer holds the Arduino Nano 33 BLE Sense at the center, while one motor driver is placed on each side of the Arduino and connected to its corresponding motor. A large portion of the wiring is also located on the upper layer, including signal wires between the Arduino, motor drivers, encoders, and power connections. The lower layer holds the battery box, which provides power for the robot.

This placement was chosen to improve balance, wiring reliability, and ease of testing. The motors were placed at the bottom so that the wheel axle acts as the main pivot point for balancing. The Arduino was placed near the center of the upper layer so that the IMU could measure the tilt of the main body more accurately. The motor drivers were placed close to their corresponding motors to shorten the motor wires and make the wiring more organized. The battery box was placed on the lower layer because it is one of the heavier components, and placing it lower helps reduce the center-of-mass height and improves stability. Table 1 summarizes the placement of the main robot components and explains how each placement decision supports balance, wiring organization, and overall system stability.

Component	**Placement**	**Reason**
DC motors and wheels	Lowest part of the robot, left and right sides	Provide movement and balance correction
Arduino Nano 33 BLE Sense	Center of the upper layer	Measures the robot tilt angle and runs the control program
Left motor driver	Left side of the upper layer, near the left motor	Drives the left motor and reduces wire length
Right motor driver	Right side of the upper layer, near the right motor	Drives the right motor and reduces wire length
Battery box	Lower layer of the chassis	Keeps heavy mass lower and improves stability
Wires	Mostly on the upper layer and between layers	Connects Arduino, motor drivers, encoders, motors, and power supply

Table 1. Summary of Component Placement and Design Purpose

2.1.3 Center-of-Mass Considerations

The center of mass is an important factor in the robot's balancing performance. Since the robot balances around the wheel axle, the mass should be arranged as symmetrically as possible around the centerline of the robot. If the battery or other heavy components are placed too far forward or backward, the robot may constantly drift in that direction and require larger motor corrections. If the weight is uneven on the left and right sides, the robot may turn unintentionally or one motor may need to work harder than the other. For this reason, the final layout placed the battery and electronics close to the central horizontal axis of the chassis, while keeping the left and right sides as balanced as possible.

2.1.4 Photos and Diagrams

Figure 4 shows the top view of the final robot mechanical layout showing the two-layer structure, Arduino placement, motor drivers, wiring, motors, and wheels.

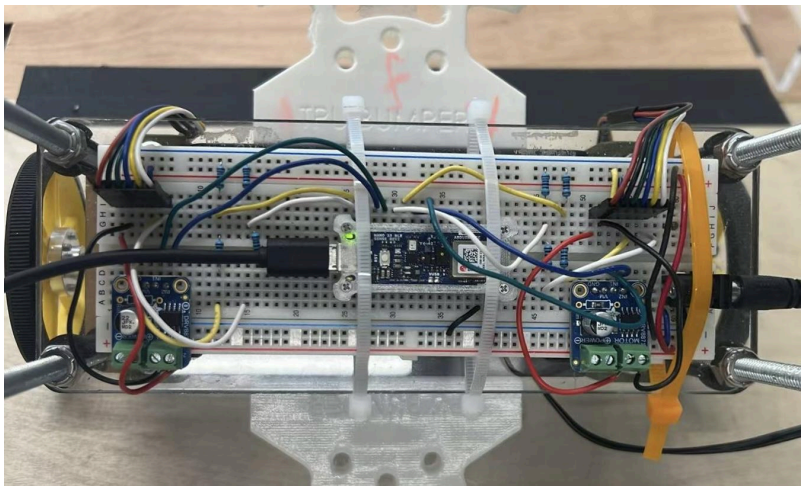


Figure 4. Top view of the final robot mechanical layout

2.2 Electrical Design

2.2.1 Circuit schematics

The physical wiring layout is shown in Figure 4. This figure shows the actual placement of the Arduino, motor drivers, motor connections, encoder wires, and power wiring on the upper layer of the robot. The electrical system connects the Arduino Nano 33 BLE Sense, motor drivers, DC motors, encoders, and battery power supply. The Arduino Nano 33 BLE Sense acts as the main controller of the robot. It reads feedback from the onboard IMU and wheel encoders, runs the control algorithm, and sends control signals to the motor drivers. Since the Arduino cannot directly provide enough current to drive the motors, the motor drivers are used as the interface between the Arduino and the DC motors.

Each motor driver receives direction and PWM control signals from the Arduino and supplies the required voltage and current to its corresponding motor. This allows the Arduino to control both the speed and direction of the wheels for balancing and movement. The encoders send wheel rotation feedback back to the Arduino, which is used to estimate wheel position and velocity. The battery provides power to the robot system, and all modules share a common ground so that the control and feedback signals use the same voltage reference.

2.2.2 Wiring diagram

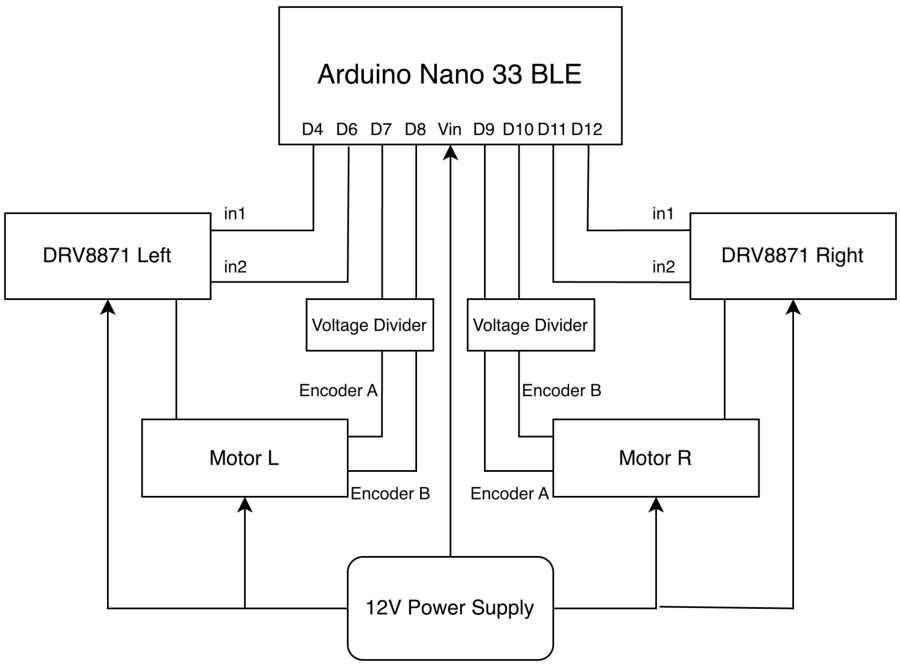


Figure 5. Wiring diagram of the self-balancing robot electrical system.

Figure 5 shows the wiring diagram of the final self-balancing robot electrical system. The Arduino Nano 33 BLE Sense is used as the central controller and is connected to two DRV8871 motor drivers, one for each motor. The Arduino sends control signals to the IN1 and IN2 pins of each motor driver, allowing the software to control the direction and speed of the left and right motors. The motor drivers receive power from the 12 V supply and provide the required motor current to drive the DC motors.

The wheel encoders are connected back to the Arduino to provide feedback on wheel rotation. Encoder A and Encoder B signals are passed through voltage dividers before reaching the Arduino input pins, which protects the 3.3 V logic pins on the Arduino Nano 33 BLE Sense. The 12 V power supply provides power to the motors and the controller through the appropriate power connections. All parts of the system share a common ground so that the motor control signals and encoder feedback signals have the same voltage reference.

2.2.3 Power distribution

The robot uses a 12 V power supply as the main power source. The 12 V supply provides motor power to the two DRV8871 motor drivers, which then deliver the required voltage and current to the left and right DC motors. The Arduino Nano 33 BLE Sense is powered through the appropriate input power connection and sends only low-power control signals to the motor drivers. This separates the high-current motor power path from the low-power control path, preventing the Arduino pins from directly driving the motors.

Because the Arduino Nano 33 BLE Sense uses 3.3 V logic, the encoder feedback signals cannot exceed the safe input voltage of the Arduino pins. To protect the Arduino, each encoder signal is passed through a voltage divider before reaching the Arduino input pins. A 27 kΩ resistor and a 10 kΩ resistor were used to reduce the signal voltage from 12 V to approximately 3.24 V. All components, including the Arduino, motor drivers, encoders, and power supply, share a common ground so that the PWM control signals and encoder feedback signals have the same voltage reference.

2.2.4 Sensors and actuators

The main sensors used in the robot are the gyroscope and accelerometer on the Arduino Nano 33 BLE Sense and the wheel encoders connected to the motors. The gyroscope and accelerometer measure the robot's tilt angle, which is the key feedback signal for the balancing control loop. The encoders measure wheel rotation and provide feedback for estimating wheel position and velocity. These sensor signals allow the robot to adjust its motor output based on both body angle and wheel movement.

The actuators in the system are the two DC motors, one on the left side and one on the right side of the robot. Each motor is controlled through a DRV8871 motor driver. By changing the PWM and direction signals sent to the motor drivers, the Arduino can control the speed and direction of each wheel. This allows the robot to correct its tilt, move forward or backward, and turn left or right during BLE control.

2.3 Software and Control Design

2.3.1 Software architecture

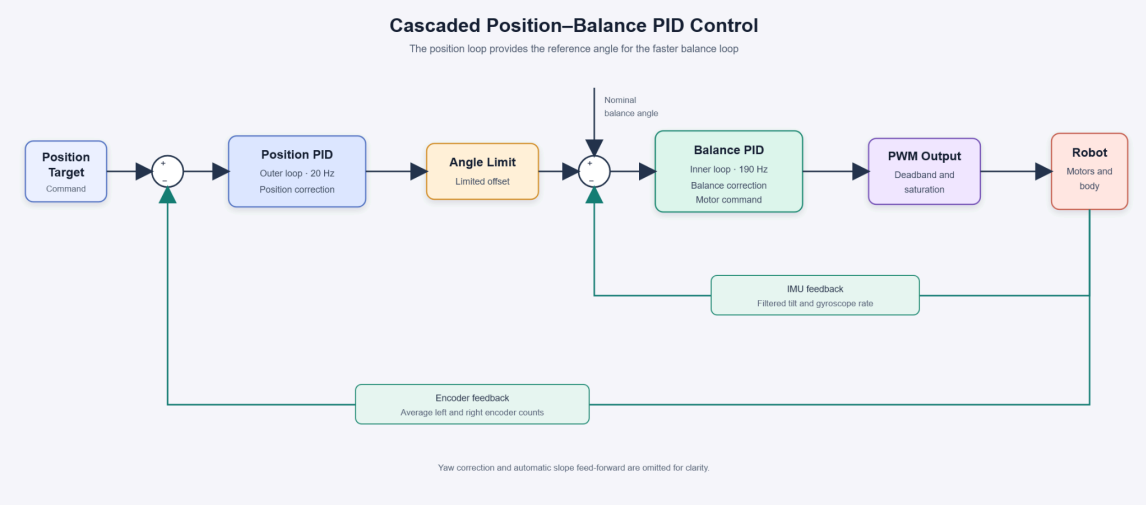


Figure 6. Cascaded PID Control

The software architecture was designed as a real-time control system running on the Arduino Nano 33 BLE Sense. The program is organized into several main parts: IMU sensor reading, angle estimation, encoder feedback, cascaded PID control, motor output, and BLE communication.

When the robot starts, the software initializes the motor driver pins, encoders, BLE connection, serial communication, and BMI270 IMU. A gyroscope calibration routine is performed before balancing so that the pitch and yaw gyro offsets can be measured and removed from later readings. This helps reduce sensor drift and improves the accuracy of the angle estimation.

The main control loop runs at a fixed timing rate. In each cycle, the robot reads the IMU data, estimates the tilt angle, checks safety conditions, calculates the control output, and sends PWM signals to the motor drivers. The encoder counts are updated using interrupts, allowing the software to track wheel movement without delaying the main loop.

The controller uses a cascaded PID structure as shown in Figure 6. The outer position loop runs at a slower rate, about $\frac{1}{10}$ of the inner angle loop frequency, and compares the target position with the measured wheel position from the encoders. Its output is a small target angle offset. This target angle is then sent to the faster inner angle loop, which keeps the robot balanced by adjusting the motor PWM based on the tilt angle and gyro rate.

2.3.2 Sensor processing and filtering

The robot uses the onboard BMI270 IMU and two wheel encoders as the main feedback sensors. The IMU provides accelerometer and gyroscope data for estimating the tilt angle of the robot, while the encoders measure the wheel displacement for the outer position loop. The accelerometer and gyroscope are configured to run at a 400 Hz sample rate so that the inner balance loop can receive updated angle feedback quickly.

Before the control system starts, the software performs an automatic gyroscope calibration. During this step, the robot is kept still while the program collects stable IMU samples. The average pitch gyro value is stored as the gyro offset and subtracted from later readings. This reduces drift caused by sensor bias. A yaw gyro offset is also calculated for yaw correction.

The tilt angle is first estimated from the accelerometer using the gravity direction:

$$\theta_A = tan^{-1}\left(\frac{ay}{az}\right) \times \frac{180}{\pi}$$

In order to prevent the influence of the noise during the robot's vibration and oscillation. A complementary filter is used:

$$\theta_N = K\,(\theta_N + \omega\Delta t) + (1 - K)\theta_A$$

Where K = 0.98, ω is the calibrated gyro rate and Δt is calculated from the actual loop timing

In this filter, the gyroscope provides most of the short-term angle response, while the accelerometer slowly corrects long-term drift.

Encoder signals are processed using interrupt routines so that wheel counts are updated immediately when the encoder state changes. The left and right encoder counts are averaged to calculate the robot's measured position:

$$Position = \frac{CountL + CountR}{2}$$

2.3.3 Control algorithms

The robot uses a cascaded PID control algorithm to achieve both self-balancing and position control. The control system is divided into an outer position loop and an inner angle loop. The position loop runs every 20 Hz, while the inner angle loop runs at about 190 Hz. Therefore, the outer loop runs at approximately one-tenth of the speed of the inner loop.

The outer position loop compares the target wheel position with the measured position from the encoders:

$$e_p = x_{target} - x_{measured}$$

The average encoder count from the left and right wheels is used as the measured position. The position controller then calculates a target angle offset based on the standard PID equation:

$$\theta_{offset} = K_p \times e_p + K_i \times \int e_p dt - K_d \times V_p$$

Where V_p is the measured wheel velocity

Vp is calculated based on the encoder:

$$V_p = \frac{counts\,(k) - counts(k-1)}{dt}$$

The output of this loop is limited to a safe range so that the robot does not lean too aggressively. In the final implementation, the maximum position angle offset was limited to 4°.

The target angle for the inner balance loop is calculated by adding this position angle offset to the base balance angle:

$$\theta_{target} = \theta_{base} + \theta_{offset}$$

where the base target angle was set to $-\ 0.5^{\circ}$ to compensate for the robot's mechanical balance point.

The inner angle loop is responsible for keeping the robot upright. It compares the target angle with the filtered tilt angle from the IMU:

$$e_\theta = \theta_{target} - \theta_{measured}$$

The balance controller then calculates the motor command using PID control:

$$PWM = K_p \times e_\theta + K_i \times \int e_\theta dt - K_d \times \omega_\theta$$

Where ω_θ is the pitch gyro rate

The derivative term uses the gyro rate directly instead of differentiating the angle error, which helps reduce noise. Table

Several protection and compensation features are included in the control algorithm. Integral limiting is used to prevent excessive integral buildup, and the final PWM command is clamped between −255 and 255. A motor deadband compensation is also added so that small control outputs can still overcome motor friction. In addition, yaw correction is applied by slightly adjusting the left and right motor commands to reduce unwanted rotation.

2.3.4 BLE integration

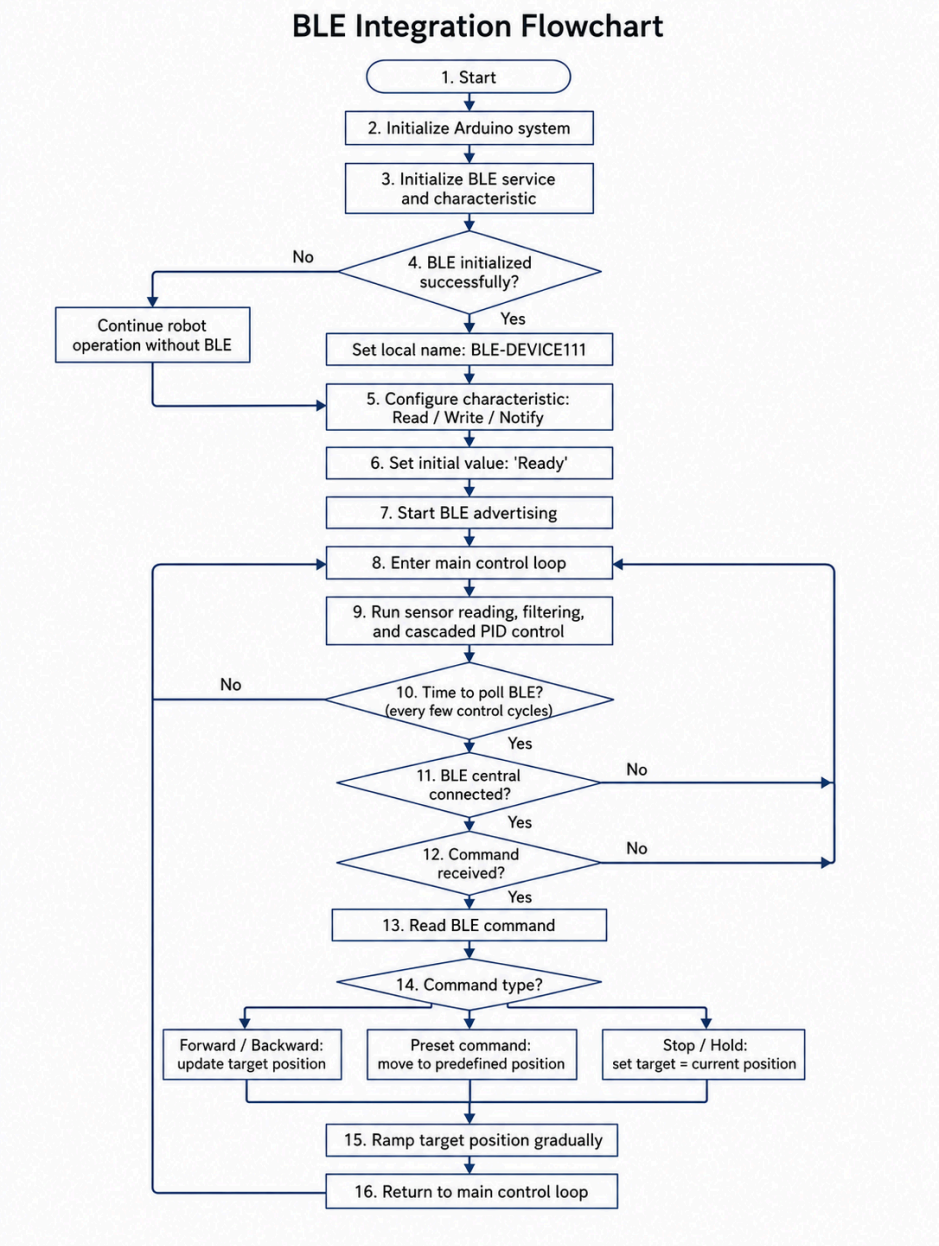


Figure 7. BLE Flowchart

The process of the BLE control is shown in Figure 7. The program uses the ArduinoBLE library to create a custom BLE service and characteristic. The robot advertises with the local name “BLE-DEVICE111”, allowing it to be detected and connected by an external BLE device.

The BLE characteristic is configured with read, write, and notify permissions. At startup, the characteristic is initialized with the message "Ready" and the robot begins advertising. If BLE initialization fails, the robot can still continue running without BLE, which prevents communication errors from stopping the main balance controller.

To avoid slowing down the real-time control loop, BLE is not checked continuously in a blocking way. Instead, the program polls BLE only every few control cycles, after the main control calculation is completed. This keeps BLE communication separate from the high-speed balance loop and helps maintain stable control timing.

When a BLE command is received, the software reads the first character of the command and updates the robot’s target position. A stop or hold command sets the target position to the robot’s current encoder position, allowing it to hold its place.

The BLE target position is also ramped instead of being changed instantly. This means the target position moves gradually toward the commanded value, which prevents sudden changes in the position loop and reduces aggressive leaning.

2.3.5 Final control gains and parameters

As can be seen in Table 2 and Table 3, the parameters of the control loops are relatively conservative with higher control frequency.

Angle Loop

Parameter	Value
Kp	8.5
Ki	65
Kd	0.5
Base target angle	-0.5°
Maximum safe angle	40°
Balance integral limit	255 PWM
Loop Frequency	190Hz

Table 2. Angle Loop Parameters

Position Loop

Parameter	Value
Kp	0.0016
Ki	0
Kd	0.0007
Position loop frequency	$20Hz$
Maximum angle offset	4°

Table 3. Position Loop Parameters

3. Design Evolution and Engineering Decisions

3.1 Midpoint Demo Performance

At the midpoint demo, the robot could balance in place for a short time, but the result was not stable or repeatable enough for the final demo. The robot still needed manual calibration, and the balance result depended on how accurately the robot was positioned before startup. It also had noticeable drift when it was supposed to stay stationary.

The BLE control was already working at this stage. The robot could receive forward, backward, left, and right commands, but the movement was slow and not very accurate. During forward and backward movement, the robot did not travel straight because the two motors did not respond exactly the same. The robot also could not complete the ramp task at this point.

Because of these problems, the design after the midpoint demo focused on four main areas: improving sensor mounting, changing the calibration method, improving the control structure, and compensating for motor differences.

3.2 IMU Mounting and Calibration

One important change after the midpoint demo was improving the IMU mounting. During testing, we found that the Arduino board and IMU could move slightly on the breadboard when the robot vibrated. Since the balance controller depends directly on

the measured angle, even a small movement of the IMU could affect the controller output. To reduce this problem, the board was secured more firmly with zip ties so that the IMU stayed fixed relative to the robot body.

The calibration method was also changed from manual calibration to automatic calibration. Manual calibration was simple, but it depended too much on how the robot was placed before startup. A small reference angle error could cause the robot to drift forward or backward, even with no BLE command. Automatic calibration was chosen because it made startup more repeatable and reduced the effect of human error.

An alternative approach was to keep manual calibration and only tune the PID gains more carefully. This was not selected because the same PID values could behave differently if the starting angle was not consistent. Fixing the calibration method first made the controller easier to tune.

3.3 Control Structure Update

The control structure shown in Table 4 was updated to make the robot more controllable during both stationary balancing and BLE movement. The final design used IMU data with a complementary filter to estimate the robot tilt angle. This angle was used by the balancing PID loop, which generated the main PWM correction for keeping the robot upright.

For forward and backward motion, encoder feedback was added as an outer position loop. Instead of directly setting the motor PWM, the position loop adjusted the target angle of the balancing loop. This allowed the robot to move by leaning slightly forward or backward while still using the balance loop to stay upright.

The final control system used three loops. The balancing loop was the inner loop. The position loop and turning loop were outer corrections added on top of the balancing controller.

Control loop	Controller type	Main purpose
Balancing loop	PID	Keep the robot upright
Position loop	PD	Control forward and backward distance
Turning loop	P	Control left and right rotation

Table 4: Final control loop structure.

The position loop was written in PID form, but the final integral gain was set to zero. Therefore, the final position controller behaved as a PD controller. The turning loop used only proportional control.

Another approach we considered was directly commanding motor PWM for forward and backward movement. This was easier to implement, but it did not control distance accurately and made the robot very sensitive to motor mismatch. For this reason, the encoder-based position loop was used in the final design.

3.4 Motor and BLE Improvements

Motor behavior was another major reason for design changes. After the robot was fully assembled, the motor deadband was measured again. The final measured deadband values were 41 for the left motor and 42 for the right motor in Table 5. These values were used because small PWM outputs below the deadband could not move the motors reliably.

Motor	Final deadband value
Left motor	41
Right motor	42

Table 5: Final measured motor deadband values.

Deadband compensation helped the robot respond to smaller controller outputs. Without this compensation, the controller could calculate a PWM correction, but the motor might not move until the error became larger. This caused delayed correction and made the robot easier to destabilize.

The BLE system was also improved. During testing, repeated commands sometimes caused unwanted behavior, especially for distance movement and turning. A command handling mechanism was added so that one BLE input produced one controlled action instead of being triggered multiple times accidentally.

An alternative approach was to process every BLE input immediately. This was not used because it could disturb the control loop timing and make the robot response less predictable. The final BLE handling was slower but more reliable for the demo tasks.

3.5 Mechanical Adjustment

A mechanical change was also made to the battery holder. The battery position affected the center of mass of the robot. If the battery moved during testing, the balance point

changed, and the same PID gains did not work as well. The battery holder was reinforced so that the battery stayed in a more consistent position.

This change was important because the controller was tuned for one physical balance point. If the center of mass shifted, the robot could start drifting or require a different target angle. The mechanical adjustment helped improve repeatability between tests.

3.6 Final Design Direction

By the final demo, the robot had changed from a short-duration balancing prototype into a BLE-controlled self-balancing robot. The final design combined IMU feedback, encoder feedback, motor deadband compensation, BLE command handling, and a three-loop control structure.

The main engineering decision was to not treat the robot as only a PID tuning problem. The final performance depended on both software and hardware details, including IMU mounting, calibration, motor deadband, motor mismatch, BLE timing, and battery placement.

4. Testing, Tuning, and Performance Analysis

The final robot was tested in stages instead of tuning everything at once. This was necessary because the robot used three connected control loops. The balancing loop had to work first. After that, the position loop could be tuned for forward and backward movement. Finally, turning and ramp behavior could be tested under BLE control.

Three main tests were used during the project:

1. Balance and sensor stability test
2. Motor deadband and position accuracy test
3. Turning and ramp performance test

4.1 Balance and Sensor Stability Test

The first test focused on stationary balance and IMU stability. The purpose was to check whether the robot could stay upright without receiving a movement command. During this test, we observed how long the robot could balance, whether it drifted forward or backward, whether it oscillated, and whether the angle reading changed unexpectedly during vibration.

At the midpoint stage, the robot could balance for a short time, but it was not repeatable. Sometimes it drifted even when no command was sent. In other trials, it

slowly tilted until it fell. This showed that the problem was not only the PID gains. The sensor reference and mechanical mounting also affected the result.

During testing, we found that the Arduino board and IMU were not fixed firmly enough. Motor vibration could cause small movement of the board on the breadboard. Since the controller used the IMU angle as its main feedback signal, this movement made the controller react to sensor movement instead of the actual body tilt. The board was then secured with zip ties, and the calibration method was changed from manual calibration to automatic calibration.

After the mounting and calibration were improved, the balancing loop was tuned again. Table 6 shows the final balancing PID gains.

Term	Final value	Function
Kp	8.5	Main tilt correction
Ki	65	Drift and bias correction
Kd	0.5	Damping and overshoot reduction

Table 6: Final balancing loop PID gains.

The gains were tuned based on the robot's behaviour. If Kp was too low, the robot corrected too slowly and fell. If Kp was too high, the robot oscillated. The derivative term helped reduce fast oscillation and overshoot. The integral term was used to reduce the slow drift caused by small bias errors. After tuning, the robot became more stable and repeatable in stationary mode.

4.2 Motor Deadband and Position Accuracy Test

The second test focused on motor deadband and forward/backward distance control. At the midpoint demo, the robot could move under BLE control, but it moved slowly and did not travel straight. The two motors did not produce the same motion for the same PWM command, so the robot tended to turn instead of moving in a straight line.

To test this, the motor deadband was measured after the robot was fully assembled. Deadband is the PWM range where a command is sent to the motor but the motor does not move yet. This matters because small PID outputs may be too weak to overcome static friction. If the deadband is not handled, the controller response becomes delayed.

The final measured deadband values are shown in Table 7.

Motor	Final deadband value
Left motor	41
Right motor	42

Table 7: Final motor deadband measurements.

These values were added to the motor mapping. After deadband compensation, small control outputs could move the wheels more reliably. This made the balancing response smoother and also helped forward/backward movement.

After motor mapping was updated, the position loop was tuned for the 50 cm forward and backward BLE tasks. The final position loop gains were Kp = 0.0016, Ki = 0.0000, and Kd = 0.0007 shown in Table 8. Although the loop was implemented in PID form, the integral gain was set to zero, so the final controller worked as a PD controller.

Term	Final value	Function
Kp	0.0016	Correct distance error
Kd	0.0007	Reduce overshoot near the target

Table 8: Final position loop gains.

The proportional term corrected the distance error, and the derivative term helped reduce overshoot near the target. Integral control was not used because accumulated position error could make the robot continue moving after reaching the target.

The final position test showed that the robot could move forward 50 cm and backward 50 cm and stop within ±3 cm of the target. This met the stopping accuracy target. The result showed that distance control required both encoder feedback and motor deadband compensation. Encoder feedback alone was not enough if the motor outputs were not mapped properly.

4.3 Turning and Ramp Performance Test

The third test focused on turning and ramp performance. This test was done after the stationary balance and position control were working, because turning and ramp movement both depend on stable balancing.

For turning, the robot used a P-only turning loop with a final turning gain of Kp = 0.4. The turning loop changed the PWM difference between the left and right motors. A derivative term was not used because the yaw signal during rotation was noisy, and

derivative control could amplify this noise. A proportional-only controller gave a simpler and smoother turning response in Table 9.

Term	Final value	Function
Turning Kp	0.4	Generate left-right PWM difference for turning

Table 9: Final turning loop gain.

The robot was tested using two 45° turns to complete a 90° turn. Each 45° turn was completed in less than 2 seconds. The left 90° turn reached the ±3° target, but the right 90° turn had about 10° of final heading error. This showed that the turning loop worked, but the robot did not turn symmetrically in both directions. The likely reasons were motor mismatch, wheel slip, friction difference, and yaw estimation error.

The ramp test was done at 5°, 10°, and 15°. The robot successfully completed the 5° ramp task, including moving up the ramp, stopping on the ramp, and moving down while staying balanced. For the 10° and 15° ramps, the robot could climb up, but it could not reliably come down while maintaining control. This showed that downhill motion was harder than uphill motion because gravity increased the wheel speed and the controller had to balance while also slowing the robot down.

The final performance results are summarized in Table 10.

Task	Final result	Target or requirement	Status
Forward movement	50 cm within ±3 cm	±3 cm stopping accuracy	met
Backward movement	50 cm within ±3 cm	±3 cm stopping accuracy	met
45° turn	Less than 2 s	Less than 2 s	met
Left 90° turn	Within ±3°	±3° target	met
Right 90° turn	About 10° error	±3° target	Not met
5° ramp	Up, stop, and down	Baseline ramp task	met
10° and 15° ramps	Up, stop,	Optimization task	Partially met

Table 10: Final performance test summary.

Overall, the testing process showed that the robot had to be tuned in order. The balance loop had to be stable before the position loop could be tuned. After that, turning and

ramp control could be tested. This helped separate sensor problems, motor problems, and controller problems instead of changing all parameters at the same time.

5. Challenges and Troubleshooting

5.1 Stationary Drift and Inconsistent Balancing

One major challenge was that the robot could balance for a short time but could not always stay stationary. At the midpoint stage, the robot often drifted forward or backward even when no BLE movement command was given. In some tests, it stayed upright for a few seconds and then slowly moved away from the starting point. In other tests, it gradually tilted until it fell.

The first suspected cause was a calibration error. The robot originally used manual calibration, so the balance reference depended on how the robot was positioned before startup. If the reference angle was slightly wrong, the controller would drive the motors even when the robot was supposed to stay still.

Another suspected cause was IMU movement. The Arduino board and IMU were mounted on the breadboard, and vibration from the motors could move the board slightly. This changed the measured angle and made the balance loop react incorrectly.

The troubleshooting process started with the mechanical and sensor setup before changing the PID gains. The IMU board was secured more firmly with zip ties. After that, automatic calibration was added to make startup more repeatable. Once the sensor reading became more consistent, the balancing PID was tuned again. The final balance loop used Kp = 8.5, Ki = 65, and Kd = 0.5.

The final solution was not one single PID change. It was a combination of better IMU mounting, automatic calibration, and balance loop tuning. The robot still had small motion during balancing, but it became reliable enough for the final demo tasks. This showed that stationary drift is not always only a controller problem. The sensor mounting and physical balance point must also be stable.

5.2 Motor Mismatch and Asymmetric Turning

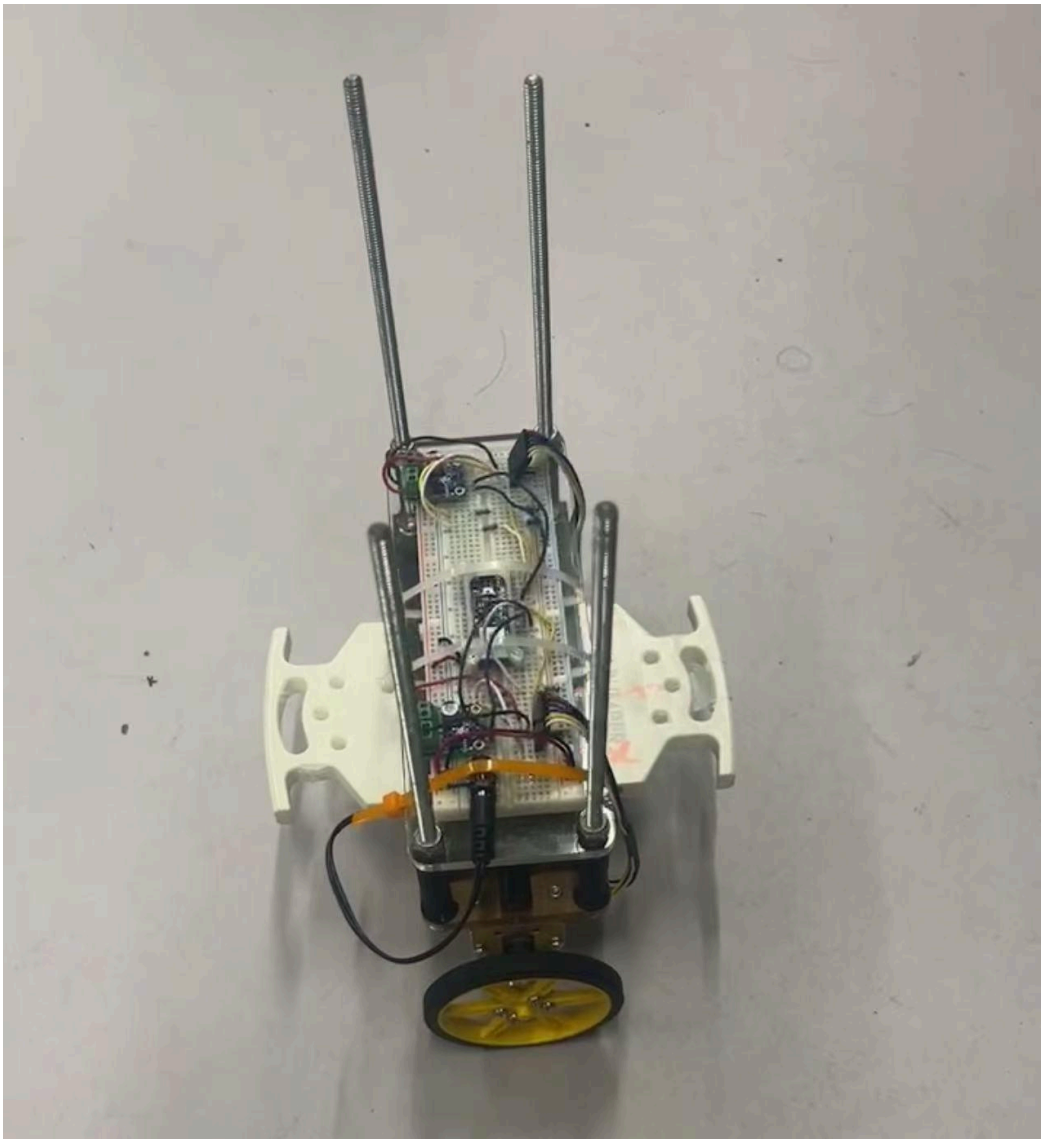


Figure 8. Asymmetric Performance of the Robot

Another major challenge was that the left and right motors did not respond the same way. During forward and backward BLE movement, the robot did not always travel straight. This also affected turning. In the final demo, the left 90° turn reached the ±3° target, but the right 90° turn still had about 10° of error.

The suspected causes were motor deadband difference, motor strength mismatch, wheel friction, and wheel slip. Even if the same PWM value was sent to both motors, the actual wheel speeds were not exactly the same, as can be seen in Figure 7. This made both straight movement and turning harder to control.

To troubleshoot this issue, the motor deadband was measured again after the robot was fully assembled. The final measured values were 41 for the left motor and 42 for the

right motor. These values were used in the motor mapping so that small controller outputs could move the motors more reliably.

Encoder feedback was also used in the position loop to improve forward and backward movement. For turning, a P-only turning loop was used to apply a difference between the left and right motor PWM outputs.

The final result was good for straight-line distance control. The robot could move forward and backward 50 cm and stop within ±3 cm of the target. It also completed each 45° turn in less than 2 seconds. However, the right-turn accuracy was still worse than the left-turn accuracy. This means the basic turning loop worked, but the robot still needed direction-specific turning calibration.

The main lesson from this problem was that the two motors should not be treated as identical. Separate motor deadband values helped, but they did not fully remove the left-right difference.

5.3 Steep Ramp Descent Failure

The third major challenge was ramp performance, especially when the robot came down steeper ramps. At the midpoint stage, the robot could not complete the ramp task. By the final demo, the robot could complete the 5° ramp task, but it could not reliably descend the 10° and 15° ramps.

The suspected cause was that climbing and descending required different control behavior. When the robot climbed up the ramp, the motors mainly needed enough torque to move upward while the balance loop kept the body upright. When the robot descended, gravity pulled it downhill and increased the wheel speed. This made the robot harder to control because the controller had to keep the robot balanced while also preventing it from rolling too fast.

The troubleshooting process was done by testing the ramp task at different angles. The 5° ramp was tested first because it was the baseline requirement. After the robot could complete the 5° ramp, 10° and 15° ramps were tested. The robot could climb the steeper ramps, which showed that motor torque was not the main limitation. The failure happened more often during descent, so the problem was mainly downhill speed control.

The final solution was to make the 5° ramp reliable for the final demo. The 10° and 15° ramp results were treated as partial optimization because the robot could climb them but could not descend them consistently.

This challenge showed that ramp climbing and ramp descending should not use exactly the same control behavior. A better future version should include a separate downhill mode with stronger speed limiting or braking compensation.

6. Conclusions

6.1 Final System Performance

The final robot performed much better than the midpoint version. At the midpoint demo, the robot could only balance for a short time, needed manual calibration, had noticeable drift, moved slowly under BLE control, and could not complete the ramp task. By the final demo, the robot could self-balance more reliably, respond to BLE commands, move forward and backward, turn left and right, and complete the baseline ramp task.

For forward and backward movement, the robot completed the 50 cm BLE tasks and stopped within ±3 cm of the target. For turning, each 45° turn was completed in less than 2 seconds. The left 90° turn met the ±3° target, while the right 90° turn had about 10° of final error. For the ramp task, the robot completed the 5° ramp by moving up, stopping, and moving down while maintaining balance. It could also climb the 10° and 15° ramps, but it could not reliably descend them.

Overall, the robot met the main baseline requirements and achieved some optimization goals. The main remaining limitations were right-turn accuracy and downhill control on steeper ramps.

6.2 Technical Achievements

The main technical achievement was building a complete closed-loop self-balancing robot using IMU feedback, encoder feedback, BLE communication, and motor PWM control. The final system used a complementary filter to estimate the tilt angle, a PID balancing loop with gains Kp = 8.5, Ki = 65, and Kd = 0.5, a PD position loop with Kp = 0.0016 and Kd = 0.0007, and a P-only turning loop with Kp = 0.4.

Several practical improvements were also important. The IMU was secured more firmly to reduce sensor movement. Automatic calibration was added to make startup more consistent. The motor deadbands were measured and compensated using final values of 41 for the left motor and 42 for the right motor. BLE command handling was also improved so that commands were not triggered repeatedly by accident.

The layered control structure made the robot easier to tune. Instead of using one controller for everything, the system separated balancing, position control, and turning correction.

6.3 Important Lessons Learned

The most important lesson was that PID tuning alone was not enough to make the robot work well. The robot behavior was affected by sensor mounting, calibration, motor deadband, motor mismatch, BLE command handling, and battery placement. If any of these changed, the same PID gains could produce different results.

Another lesson was that the robot had to be tuned in order. The balance loop had to be stable first. Then the position loop could be tuned for forward and backward movement. After that, turning and ramp performance could be tested. Trying to tune all parts at the same time made it harder to identify the real cause of a problem.

The project also showed that different tasks require different control priorities. Stationary balancing mainly depends on a stable angle estimate and balance loop tuning. Distance movement needs encoder feedback. Turning needs left-right motor correction. Ramp descent needs stronger speed control than ramp climbing.

7. Future Improvements

7.1 Improved Downhill Ramp Control

The ramp task could be improved by adding a separate downhill control mode. In the final demo, the robot completed the 5° ramp task and could climb the 10° and 15° ramps, but it could not reliably descend the steeper ramps. This means the motors had enough torque to climb, but the controller did not control downhill speed well enough.

A future version could limit the wheel speed more strongly during descent and add braking compensation when the robot starts rolling too fast. The position or velocity loop could also use different gains on ramps compared with flat ground. This would help the robot stay balanced while preventing it from accelerating downhill.

7.2 Direction-Specific Turning Calibration

Another important improvement is to tune left and right turns separately. In the final demo, the left 90° turn reached the ±3° target, but the right 90° turn had about 10° of error. This shows that the robot did not rotate symmetrically in both directions.

A future version could use separate turning gains or separate output compensation for left and right turns. This would help compensate for motor mismatch, wheel friction, and different slip behavior in each turning direction. Another improvement would be to combine yaw feedback with encoder feedback during turns, so the robot can check both body heading and wheel movement.

7.3 Better Mechanical Stability

Mechanical stability should also be improved. During the project, IMU movement and battery position both affected the robot's balance behavior. Although the IMU and battery holder were reinforced in the final version, a more rigid mounting design would make the robot more repeatable.

A future robot could use a fixed custom mount for the Arduino/IMU board instead of relying on the breadboard and zip ties. The battery holder could also be redesigned so that the battery cannot shift during motion or ramp testing. This would keep the center of mass more consistent and reduce the amount of retuning needed after mechanical changes.

7.4 Battery Voltage and Motor Compensation

Battery voltage monitoring would also improve reliability. As the battery voltage drops, the same PWM command may produce less motor torque. This can change the robot's balance response and make the tuned controller less consistent.

A future design could measure battery voltage and adjust the motor output based on the available voltage. The motor compensation could also be expanded beyond deadband values. For example, the robot could use separate compensation values for forward and backward motor directions. This would make the robot behavior more consistent across different battery levels and motor directions.

7.5 More Systematic Data Logging

Another useful improvement would be better data logging during testing. In this project, many tuning decisions were made by observing whether the robot drifted, oscillated, overshot, or fell. This worked for basic tuning, but it was not very precise.

A future version could record angle, encoder counts, yaw angle, PWM output, and loop frequency during each test. These logs would make PID tuning more quantitative. For example, if the robot failed during ramp descent, the recorded data could show whether the problem came from excessive wheel speed, motor saturation, or angle error.

Overall, the most important future improvements are stronger downhill control, better right-turn calibration, more rigid mechanical mounting, and better test data collection. These changes directly target the main limitations observed in the final demo.